Listing 13.14 Passing parameters to an SQL statement using EF Core

```
var p1 = new SqliteParameter("@name", Name);
var sql = "select * from cities where name = @name";
var city = _context.Cities.FromSqlRaw(sql, p1).FirstOrDefault();
```

EF Core also supports passing format strings with parameter values in placeholders. When you use format strings with EF Core, the placeholders are converted to parameters at runtime:

```
var sql = "select * from cities where name = {name}";
var city = _context.Cities.FromSqlRaw(sql, Name).FirstOrDefault();
```

I tend to avoid this approach because the conversion of format string placeholders to parameters is not widely supported by other ORMs, and if you are not careful, you can inadvertently introduce an SQL injection vulnerability when swapping between one ORM and another.

13.4.2 Cross-site scripting

XSS is a type of injection attack, whereby the malicious user injects client-side script into the pages of a vulnerable website. Once the script is included in a page, it executes with the same privileges as the user, which, depending on the nature of the site, can have a serious impact. The injected code could be designed to capture keystrokes and transmit them via AJAX to a remote server, potentially exposing account numbers, usernames, passwords, and so on. It can also alter the DOM, changing the content of the page, so links take the user to an alternative site that may still look like the original site, for example.

So how does the client-side script find its way into a vulnerable site? The two most common routes are via a URL (reflected XSS) or a posted form, the contents of which are stored for later retrieval and rendering to the browser (stored XSS). Reflected XSS takes place when a bad actor creates a URL that includes a script:

```
https://vulnerable-domain.com/city?name=<script>...bad stuff..</script>
```

The victim won’t necessarily see the script in the URL when they follow the link, which could be embedded in an HTML email, for example, but if the target website renders the value of the name query parameter as is, the contents of the script block will be executed. Stored XSS happens when the bad actor uses a form to submit a script that is displayed on